Railway level crossing in mCRL2

Diogo and Rodrigo Gomes de Araújo
up202510267, up202210267

November 2026

Introduction

This report was written for the first assignment of the *Systems Verification* course at *Faculdade de Ciências da Universidade do Porto* (FCUP). It presents a model of a railway level crossing, describing the interaction between a controller, a gate, and one or more trains that wish to cross. We then formalise a set of properties that verify its correctness and check them in mCRL2.

Task 1

In this task we modelled a single train passing through the crossing.

Components

The model contains the following three processes.

$$\begin{aligned} \text{Train} &= \text{approach} \cdot \text{grant} \cdot \text{enter} \cdot \text{exit} \cdot \text{leave} \cdot \text{Train} \\ \text{Gate} &= \text{lower} \cdot \text{down} \cdot \text{raise} \cdot \text{up} \cdot \text{Gate} \\ \text{Controller} &= \text{approach} \cdot \text{lower} \cdot \text{down} \cdot \text{grant} \cdot \text{leave} \cdot \text{raise} \cdot \text{up} \cdot \text{Controller} \end{aligned}$$

Each one consists of a loop over a finite set of actions. The `Controller` enforces the ordering between the `Gate` and the `Train`'s actions. It allows the train to enter (`grant`) after the gate is `down` and `raise` the gate after the train `leaves`.

Composition

Every interaction is modelled as a pair of complementary actions (e.g. `approach_train` and `approach_controller`) that `comm` merges into one synchronised action. `allow` then ensures they occur strictly simmultaneously.

```mcrl2
hide({ lower, grant, leave, raise },
allow({ approach, lower, down, grant, leave, raise, up, enter, exit },
comm({ approach_controller | approach_train -> approach, ... },
      TRAIN || GATE || CONTROLLER )));
```

For inspection, we split the actions into messages and physical events. Messages are internal handshakes between the components and cannot be observed from the outside, so we hide them:

- **`lower`, `raise`:** the commands to the gate;
- **`grant`:** the permission to cross;
- **`leave`:** the train's notification that it left.

Physical events, on the other hand, are what someone standing near the crossing would observe, so they remain visible:

- **`approach`:** the train arriving;
- **`enter`, `exit`:** being on the crossing;
- **`down`, `up`:** the gate reaching its position.

Inspection

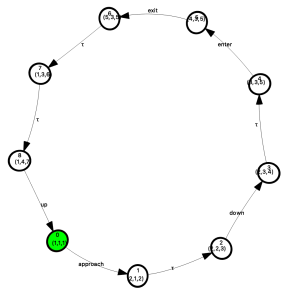


Figure 1: Full LTS

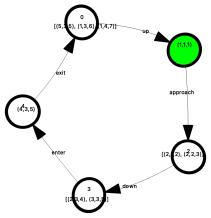


Figure 2: Reduced LTS

The full LTS (Figure 1) is a single cycle of 9 states, four of whose transitions are τ (the hidden messages). None of these τ-steps removes a choice, so branching bisimulation collapses them, leaving a cycle of 5 states made only of the physical events (Figure 2). This is the externally observable behaviour of the train passing through the crossing.

Properties

Property	Formula	Result
Safety	$[-^*;\mathrm{enter};(-\mathrm{exit})^*;\mathrm{up}]\ \mathrm{false} \wedge [(-\mathrm{down})^*;\mathrm{enter}]\ \mathrm{false} \wedge$ $[-^*;\mathrm{up};(-\mathrm{down})^*;\mathrm{enter}]\ \mathrm{false}$	true
Reachability	$[-^*;\mathrm{leave}]\langle -^*;\mathrm{raise}\rangle\ \mathrm{true}$	true
Liveness	$[-^*;\mathrm{approach}]\langle -^*;\mathrm{exit}\rangle\ \mathrm{true}$	true
Deadlock freedom	$[-^*]\langle -^*;\mathrm{approach}\rangle\ \mathrm{true}$	true

The model satisfies all four properties.

- For *safety*, the train is never on the crossing while the gate is open: once a train has entered (`enter`), the gate cannot be raised (`up`) until the train exits (`exit`). Furthermore, a train can only enter if the gate has been lowered (`down`) since it was last raised, ensuring that the gate remains closed whenever the train is on the crossing.
- For *reachability*, from any reachable state, the gate can eventually be raised (`raise`), allowing the system to return to its open state.
- For *liveness*, whenever a train approaches (`approach`), it can eventually exit (`exit`), ensuring that it is not permanently prevented from completing its passage.
- Finally, for *deadlock freedom*, from any reachable state, a new train can eventually approach (`approach`), meaning the system can continue making progress and the cycle can restart.

Broken Variant

In the faulty variant, the controller issues the grant immediately after ordering the gate to lower, without waiting for the `down` acknowledgement:

```
1  1      approach_controller                                    mcrl2
2    -    . lower_controller . down_controller
3    -    . grant_controller
   2  +    . lower_controller . grant_controller
   3  +    . down_controller
```

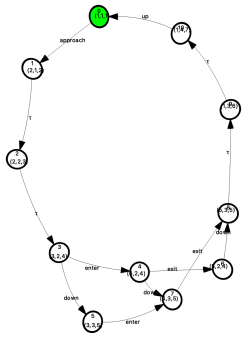


Figure 3: Broken LTS

The safety property $[(\neg\ \text{down})^* \cdot \text{enter}]$ false (no `enter` unless a `down` occurred since the last `up`) fails with the following counterexample:

approach . lower . grant . enter

Since `lower` and `grant` are hidden, the observable trace is just `approach, enter`: the train enters with no `down` before it, so the formula is violated. The gate process is still waiting to fire `down_gate`, because after `lower` the train and gate run independently and the train wins the race.

Task 2

In this task the model allows multiple trains to pass through the crossing one at a time.

Model with data

$\text{Train}(t)$ and Gate are the same as in Task 1, with the train's actions carrying its identity. The controller keeps a queue Q of the trains that approached and did not leave yet, whether the crossing c is `busy` or `free`, and the gate state g:

$$\begin{aligned}\text{Controller}(Q, c, g) = &\sum_{t \in \text{Id}} (t \notin Q) \to \text{approach}(t) \,.\, \text{Controller}(Q \triangleleft t, c, g) \\ &+ (Q \neq [\,] \wedge g = \text{closed} \wedge c = \text{free}) \to \text{grant}(\text{head}(Q)) \,.\, \text{Controller}(Q, \text{busy}, g) \\ &+ (Q \neq [\,] \wedge c = \text{busy}) \to \text{leave}(\text{head}(Q)) \,.\, \text{Controller}(\text{tail}(Q), \text{free}, g) \\ &+ (g = \text{open} \wedge Q \neq [\,]) \to \text{lower} \,.\, \text{Controller}(Q, c, \text{closing}) \\ &+ (g = \text{closing}) \to \text{down} \,.\, \text{Controller}(Q, c, \text{closed}) \\ &+ (g = \text{closed} \wedge Q = [\,] \wedge c = \text{free}) \to \text{raise} \,.\, \text{Controller}(Q, c, \text{opening}) \\ &+ (g = \text{opening}) \to \text{up} \,.\, \text{Controller}(Q, c, \text{open})\end{aligned}$$

where $Q \triangleleft t$ appends t to the end of Q. We chose a queue over a set so that only the first train in line can be granted. No train is overtaken, which matters for liveness.

Invariant. Q always holds exactly the trains that approached but did not leave yet: `approach(t)` is the only step that adds a train and `leave` the only one that removes it. c is `busy` exactly while one train is between `grant` and `leave`. Since `grant` needs $c =$ free, only one train crosses at a time, and since `raise` needs Q empty and $c =$ free, the gate is never raised while a train is near.

Properties

All properties are quantified over $t, t' \in \text{Id}$, s.t. $t \neq t'$.

Property	Formula	Result
1. Safety	$[-^*; \text{enter}(t); (-\text{leave}(t))^*; \text{up}]$ false	true

2. Mutual exclusion	$[-^*; \text{enter}(t); (-\text{exit}(t))^*; \text{enter}(t')]$ false	true
3. No premature raise	$[-^*; \text{approach}(t); (-\text{leave}(t))^*; \text{raise}]$ false	true
4. Deadlock freedom	$[-^*]\langle -^*; \text{approach}(t)\rangle$ true	true
5a. Liveness (reach)	$[-^*; \text{approach}(t); (-\text{enter}(t))^*]\langle -^*; \text{enter}(t)\rangle$ true	true
5b. Liveness (order)	$[-^*; \text{approach}(t); (-\text{enter}(t))^*; \text{enter}(t');$ $(-\text{enter}(t))^*; \text{enter}(t')]$ false	true
6. Road availability	$[-^*]\langle -^*; \text{up}\rangle$ true	true
7. Infinite opening	$[((-\text{up})^*; \text{up})^*]$ true $\wedge$ $\langle -\rangle$ true	true
8. Non-vacuity	$[-^*]\langle -^*; \text{enter}(t)\rangle$ true $\wedge$ $[-^*]\langle -^*; \text{raise}\rangle$ true $\wedge$ $[-^*]\langle -\rangle$ true	true

Safety (1–3)

The first three properties ensure that the crossing is used safely. A train can only enter when the gate is fully down (1), and only one train can occupy the crossing at a time (2). The gate can only start opening when the crossing is free and no trains are waiting. If a train approaches while the gate is opening, the gate must close again before granting access (3).

Liveness (4–5)

These properties ensure that the system continues to make progress. A train can always approach the crossing (4), and every waiting train remains able to enter (5a). The FIFO queue also prevents indefinite overtaking, since no other train can enter twice before the waiting train gets its turn (5b). Assuming the system continues to execute actions, every waiting train eventually enters.

Availability (6–8)

The gate can always eventually return to the open position (6). However, continuous train arrivals may prevent the queue from becoming empty, so recurring openings cannot be guaranteed (7). Finally, non-vacuity (8) ensures that train entry, gate raising and system transitions are reachable, making the verification meaningful.

Broken Variant

Three faulty variants are introduced to test whether the verified properties detect incorrect controller behaviour.

Naive Raise

The controller moves to the `opening` state immediately after a train leaves, without checking whether other trains are still waiting and without performing the `raise` handshake with the gate:

```
1  1      leave_controller(head(queue))                                  mcrl2
2    -    . CONTROLLER(tail(queue), free, gate)
     2 +  . CONTROLLER(tail(queue), free, opening)
```

The liveness property (5a) fails with the following counterexample:

$\text{approach}(t_1)$. lower . down . $\text{grant}(t_1)$. $\text{enter}(t_1)$. $\text{approach}(t_2)$. $\text{exit}(t_1)$. $\text{leave}(t_1)$

After $\text{leave}(t_1)$ the controller is in the `opening` state with t_2 still in the queue. It now only offers `up`, but the gate still waits for `raise`, so the two can never synchronise. The gate stays closed, `grant` requires an open-to-closed cycle that never happens again, and t_2 can no longer enter. This is the same behaviour for every train that is queued, and once all trains are queued the system deadlocks. Hence properties 4 (deadlock freedom), 5a (liveness), 6 (road availability), 7 (infinite opening) and 8 (non-vacuity, since `raise` is unreachable) fail. Property 3 still holds, as `raise` never occurs.

Missed Increment

An approaching train is not added to the queue when another train is already crossing:

```
1  1      approach_controller(t)                                  mcrl2
2    -    . CONTROLLER(queue <| t, crossing, gate)
   2  +    . CONTROLLER(queue, crossing, gate)
```

The liveness property (5a) fails with the following counterexample:

$\text{approach}(t_1)\ .\ \text{lower}\ .\ \text{down}\ .\ \text{grant}(t_1)\ .\ \text{enter}(t_1)\ .\ \text{approach}(t_2)$

Since t_2 is not recorded in the queue, the controller never grants it access. The train stays blocked at its `grant` action, so its entry is no longer reachable, even after t_1 leaves. The same happens to every train that approaches while the crossing is busy. Once all trains are lost this way, the controller can no longer synchronise on `approach` or `lower`, and the system deadlocks with the gate open. Hence properties 4 (deadlock freedom), 5a (liveness), 6 (road availability, since `up` is unreachable from the deadlocked state) and 7 (infinite opening) fail. Properties 1 to 3 still hold, as the behaviour of the trains that are queued is unchanged, and (8) holds because `enter`, `raise` and further transitions remain reachable from the initial state.

Off-by-One

The controller incorrectly updates the crossing state when a train leaves, keeping it marked as busy even though the crossing is empty:

```
1  1      leave_controller(head(queue))                           mcrl2
2    -    . CONTROLLER(tail(queue), free, gate)
   2  +    . CONTROLLER(tail(queue), busy, gate)
```

The road availability property (6) fails with the following counterexample:

$\text{approach}(t_1)\ .\ \text{lower}\ .\ \text{down}\ .\ \text{grant}(t_1)\ .\ \text{enter}(t_1)\ .\ \text{exit}(t_1)\ .\ \text{leave}(t_1)$

Although t_1 has left the crossing and the queue is empty, the controller still considers the crossing busy. Both `grant` and `raise` require $c = \text{free}$, so neither can ever happen again, and the gate stays closed. A train t_2 that approaches afterwards is queued but never granted access, since the head of the queue waits for a `leave` that its train can never produce. Once every train is stuck in the queue, no train can approach any more and the system deadlocks. Hence properties 4 (deadlock freedom), 5a (liveness), 6 (road availability), 7 (infinite opening) and 8 (non-vacuity, since `raise` is unreachable) fail. Properties 1 to 3 and 5b still hold, because neither `up` nor a second `enter` ever occurs.

State Space and Abstraction

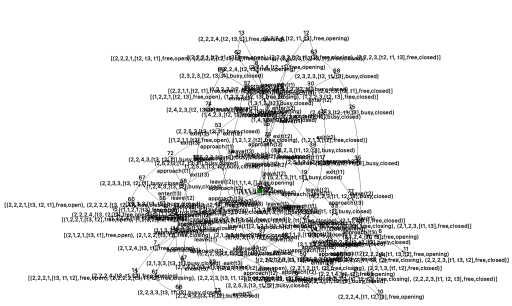


Figure 4: Full LTS

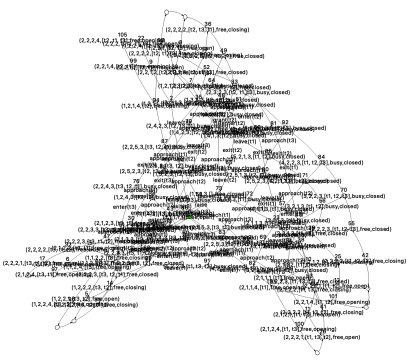


Figure 5: Reduced LTS

For $N = 1$ to 6 the full LTS has 10, 31, 108, 451, 2278 and 13695 states, respectively. The reduced LTS has 8, 23, 78, 323, 1628 and 9783. Both LTSs grow quickly with N, because the trains interleave freely and the controller keeps a queue of waiting trains.

The reduction comes from hiding the `lower` and `down` handshakes between the controller and the gate. These actions only move the gate from open to closed and are not visible to the trains. After hiding they become τ steps, and the reduction merges the states that were connected only by these steps. This removes ≈26.5% of the states, and the gain stays similar as N grows, since the reduced LTS is ≈73.5% of the full one for every N.

The trains are symmetric. They all run the same process, and the controller serves them in FIFO order, so swapping the IDs of two trains maps the LTS onto itself. The reduction preserves this symmetry, because the actions of each train keep their own labels and are never merged with those of another train.

Conclusion

In this assignment, we modelled a railway level crossing with both a single train and multiple trains, using mCRL2 to verify its behaviour. We saw how the controller coordinates the trains and the gate to ensure safe crossing and avoid conflicts. By testing faulty variants, we also understood how small mistakes in the controller can lead to safety and liveness problems, and how counterexamples help identify them. Overall, this work gave us a better understanding of how formal verification can be used to detect edge cases and prevent errors in concurrent systems.